Proyecto final

Free Fire: qué códigos de skin acepta la tienda del juego

10 % • escrito ≤ 10 pág. APA 7 • 10 diapositivas • **Python obligatorio** • plataforma UISIL + GitHub del curso

Este grupo

Grupo	5
Expo	viernes 4 de diciembre de 2026, 7:30 p.m.
Proyecto	Free Fire: qué códigos de skin acepta la tienda del juego
Excel	`P05_Skins_FreeFire.xlsx`
Integrantes	Calderón Vindas Nazareth Majenny Chaves Soto Alisson Nicole León Fallas Andrés
Plataforma	UISIL (vía oficial: escrito y diapositivas)
GitHub	`PROYECTOS_MATEMATICA_DISCRETA_USIL_2026`

Se desarrolla en Python

El modelo, la lectura del Excel y los gráficos se programan en **Python 3.10+**. El entregable es una **aplicación** que lee el Excel y emite una decisión (no un cuaderno de ejercicios). El día de la exposición el grupo ejecuta `python main.py` en vivo. Si el programa no se ejecuta, esa parte vale cero. No se aceptan capturas en lugar de la ejecución.

Entrega

Vía oficial: plataforma UISIL (PDF del escrito y PDF de las diapositivas).
Además el código se publica en el repositorio del curso:
`github.com/Franciscocamposk/PROYECTOS_MATEMATICA_DISCRETA_USIL_2026`

- Hacer *fork*, colocar todo en `Jueves/Grupo_0X_Apellido/` o `Viernes/Grupo_0X_Apellido/` (`main.py`, Excel, `README.md`, `requirements.txt`, pruebas) y enviar un *pull request*.
- Escrito $\leq$ 10 páginas, APA 7, al menos 5 fuentes. Presentación $\leq$ 10 diapositivas; **todas** las personas hablan.
- El Excel del grupo (1000 a 1700 filas) se lee con `pandas`; no se copian valores.
- El `main.py` **genera un PNG** del modelo (grafo, Hasse, AFD o barras). Sin el gráfico en la ejecución, esa parte vale cero.
- Nota 10 %: escrito 4 + código 3 + exposición 3.

PROYECTO 5 | Viernes | LENGUAJES

Free Fire: qué códigos de skin acepta la tienda del juego

5

Calderón Vindas Nazareth Majenny • Chaves Soto Alisson Nicole • León Fallas Andrés

Lenguajes Alfabeto Concatenación

Cliente. La tienda de Free Fire solo acepta skins letra mayúscula + 4 dígitos + símbolo (ejemplo: S2026#). Hay que reconocer L, no adivinar.

Problema. Sea Σ letras, dígitos y $\{\#, !, ?\}$. $L = \{X dddd\sigma : X \in \{A..Z\}, d \in \{0..9\}, \sigma \in \{\#, !, ?\}\}$. Decidir pertenencia, 5 palabras de $L \cdot L$ y el complemento en la muestra.

Datos. Hoja `Skins`: codigo, marca_oficial. Recalcular; no copiar la marca. Archivo: `P05_Skins_FreeFire.xlsx`. Carpeta en GitHub: `Viernes/Grupo_05_Apellido`.

Decisión. *¿Cuáles códigos están en L? ¿Cuántos de la muestra son hackeados? ¿Cómo se ve uno de $L \cdot L$?*



Ilustración del modelo.

Éxito. Definición de Σ y L, recuento $\in L$ / $\notin L$, 5 de $L \cdot L$ y el gráfico.

Trabajo requerido.

1. Definir Σ, L y $L \cdot L$ (tema 4.7).
2. Clasificar el Excel.
3. PNG: aceptados vs rechazados.

Marco teórico. Alfabeto, palabra, lenguaje, concatenación. Johnsonbaugh (tema 4.7).

Gráfico obligatorio: lo genera el código

El `main.py` debe generar `skins.png`: barras $\in L$ / $\notin L$ y 5 ejemplos de $L \cdot L$ impresos en la figura. En la exposición el `main.py` debe crear el PNG (matplotlib o networkx). Sin el gráfico en la ejecución, esa parte vale cero.

Se desarrolla en Python

Un reconocedor `en_L(codigo)` sin regex opaco y `skins.png`. Validación: `S2026#` $\in L$; `hack` $\notin L$. El `main.py` lee este Excel y se ejecuta en la exposición; imprime una decisión que el cliente entendería.